Práctica de Cordova / Mac y Android / 120 min

QueVer en Android

Crea una app para buscar películas y guardar tu opinión. Sigue las seis fases: reemplaza los archivos indicados y prueba cada cambio en el emulador.

Antes de empezar

Necesitas Android Studio, un emulador configurado, internet y VS Code. Usa el emulador que ya tienes. No hace falta un teléfono ni instalar Xcode.

> Comprueba que puedes instalar programas en la Mac y solicita tu clave OMDb antes de la clase. La primera compilación descarga dependencias y puede tardar.

Solicita tu clave de películas

1. Abre OMDb · API Key. Elige la opción gratuita disponible, escribe tu correo y completa el formulario.
2. Revisa tu correo y spam. Abre el enlace de activación que envía OMDb.
3. Conserva tu clave. En la fase 1 la pondrás en `www/js/config.local.js`. No la pegues en el chat, GitHub ni capturas.
4. Si el correo no llega, puedes completar la pantalla y SQLite mientras esperas; para buscar películas sí necesitarás activarla.

En cada fase

1. Abre la fase que indique el expositor.
2. Crea los archivos nuevos. En los que cambian, reemplaza **todo** el contenido.
3. En VS Code: abre el archivo → ⌘A → ⌘V → ⌘S.
4. Las líneas violetas con **+** muestran qué cambió. El botón copia el archivo completo, sin marcas.
5. Ejecuta la app y haz la prueba de esa fase.

Si te atrasas, usa los archivos de la fase actual. Conserva tu clave y no desinstales la app.

Qué hace cada archivo

`index.html`: pantalla. `index.css`: estilos. `index.js`: acciones. `db.js`: consultas SQLite. Todos están dentro de `www/`; Cordova los incluye en la app Android.

Preparar la Mac

Abrir VS Code y su terminal

1. En Finder, crea `taller-cordova` en el Escritorio. Si ya existe, comprueba que no contiene otro proyecto `quever`.
2. En VS Code abre **File → Open Folder** y selecciona `taller-cordova`.
3. Abre **Terminal → New Terminal**. Todos los comandos de la guía se ejecutan ahí, no en Debug Console ni en un archivo.

Resultado esperado: el Explorador de VS Code muestra `taller-cordova`. Abajo aparece una terminal, normalmente `zsh`, con el cursor listo.

Si falla: si no ves la terminal, vuelve a Terminal → New Terminal. Si VS Code pide confiar en la carpeta, acepta solo si es la que acabas de crear.

Ejecuta un bloque a la vez. Las salidas de ejemplo son ilustrativas: pueden cambiar rutas, versiones y tiempos. Los comandos `cd`, `mkdir` y `export` normalmente no imprimen nada.

VS Code · terminal integrada · comprobar Node

```
node --version
```

Resultado esperado: una versión compatible, por ejemplo `v22.14.0`. Node 22.9+ de la rama 22 o Node 20.17+ sirven para este taller.

Si falla: si dice `command not found`, instala Node con las instrucciones de abajo.

VS Code · terminal integrada · comprobar npm

```
npm --version
```

Resultado esperado: un número de versión, por ejemplo `10.9.2`. No tiene que coincidir con el de Node.

Si falla: npm viene con Node. Revisa su instalación y reinicia VS Code.

VS Code · terminal integrada · localizar Java 17

```
/usr/libexec/java_home -V
```

Resultado esperado: una lista de JDK instalados. Debe incluir una versión `17` y su ruta. Puede haber otras versiones.

Si falla: si no aparece Java 17 o dice que no encuentra un runtime, instala Temurin JDK 17.

VS Code · terminal integrada · comprobar Gradle

```
gradle --version
```

Resultado esperado: un informe con `Gradle`, su versión y la JVM. El Gradle del sistema no tiene que coincidir con el que descargue Cordova.

Si falla: usa "Me falta Gradle". Si el error menciona Java, configura JDK 17 en el bloque de variables y repite la consulta.

Referencia del taller: **Cordova CLI 13.0.0, Android 15.1.0 y SQLite 7.0.0**. Node 22.9 o superior dentro de la rama 22 es una opción compatible; Node 20.17+ también sirve para estas versiones. Usa **JDK 17** con Cordova, aunque Android Studio utilice otro Java.

▶ Me falta Node o Java 17

▶ Me falta Gradle

Revisar el SDK en Android Studio

Abre **Tools → SDK Manager** si tienes un proyecto abierto, o **More Actions → SDK Manager** desde la bienvenida. Confirma:

- **SDK Platforms:** Android 16 / API 36 para compilar.
- **SDK Tools:** Build-Tools 36.x, Platform-Tools, Command-line Tools (latest) y Android Emulator.
- Conserva la ruta de **Android SDK Location**. Normalmente es `~/Library/Android/sdk`.

Resultado esperado: los componentes aparecen instalados. Si marcaste alguno nuevo, pulsa Apply y espera a que termine la descarga.

Si falla: si la descarga o las licencias quedan pendientes, resuélvelas antes de compilar. No necesitas descargar otra imagen si tu emulador ya funciona.

API 36 es el SDK para compilar; el emulador no necesita tener exactamente esa versión.

Configurar la terminal de VS Code

Vuelve a la misma terminal integrada. Ajusta `ANDROID_HOME` si SDK Manager muestra otra ruta.

VS Code · terminal integrada · configurar Java y Android

```
export JAVA_HOME="$(/usr/libexec/java_home -v 17)"
export ANDROID_HOME="$HOME/Library/Android/sdk"
export PATH="$JAVA_HOME/bin:$ANDROID_HOME/platform-tools:$ANDROID_HOME/cmdline-
tools/latest/bin:$ANDROID_HOME/emulator:$PATH"
java -version
adb --version
sdkmanager --version
```

Resultado esperado: los tres `export` no imprimen nada. `java -version` indica `17`; `adb --version` muestra Android Debug Bridge; `sdkmanager --version` muestra su versión.

Si falla: revisa Java 17, la ruta del SDK y Command-line Tools (latest). Si abres otra terminal, repite este bloque; también el PATH de Gradle o Cordova si los instalaste en tu usuario.

▶ Opcional: conservar las variables para futuras sesiones

Instalar Cordova

VS Code · terminal integrada · instalar la CLI

```
npm install -g cordova@13.0.0
cordova --version
```

Resultado esperado: npm termina la instalación y la segunda línea muestra `13.0.0`. Pueden aparecer avisos de paquetes; no son lo mismo que un error.

Si falla: si ves `EACCES` o no reconoce `cordova`, consulta permisos de npm. No agregues `sudo` sin revisar la causa.

`npm` descarga Cordova; `-g` instala el comando para usarlo en distintas carpetas.

Crear el proyecto

Usa la misma terminal de VS Code. Si `taller-cordova` ya contiene otro `quever`, no lo sobrescribas: consulta al expositor para elegir otra carpeta.

VS Code · terminal integrada · crear quever/

```
mkdir -p ~/Desktop/taller-cordova
cd ~/Desktop/taller-cordova
cordova create quever com.ejemplo.quever QueVer
cd quever
```

Resultado esperado: `mkdir` y `cd` no muestran mensajes. Cordova informa que creó el proyecto. En el Explorador aparece `quever/` con `config.xml`, `package.json` y `www/`. La terminal queda dentro de `quever`.

Si falla: si indica que la carpeta ya existe o no está vacía, no la borres. Si el último `cd` falla, revisa primero si Cordova terminó de crearla.

Qué hace el comando

`create` genera la app; `quever` es la carpeta; `com.ejemplo.quever` es el identificador Android; `QueVer` es el nombre visible.

VS Code · terminal integrada · dentro de quever/

```
pwd
ls
```

Resultado esperado: `pwd` muestra una ruta que termina en `/taller-cordova/quever`. `ls` incluye `config.xml`, `package.json` y `www`.

Si falla: vuelve a entrar con `cd ~/Desktop/taller-cordova/quever`. No ejecutes los siguientes comandos desde otra carpeta.

VS Code · terminal integrada · añadir Android y SQLite

```
cordova platform add android@15.1.0
cordova plugin add cordova-sqlite-storage@7.0.0
cordova requirements android
```

Resultado esperado: Cordova añade la plataforma y el plugin. Aparecen `platforms/android/` y `plugins/`. El diagnóstico final indica `installed` para Java JDK, Android SDK, Android target y Gradle.

Si falla: corrige el requisito que marque como ausente. Si menciona licencias, ejecuta el siguiente bloque y repite `cordova requirements android`.

VS Code · terminal integrada · solo si faltan licencias

```
sdkmanager --licenses
```

Resultado esperado: aparecen licencias para revisar, o un aviso de que ya están aceptadas. Al aceptar las pendientes, el proceso termina sin error.

Si falla: si no reconoce `sdkmanager`, revisa Command-line Tools y el PATH de la terminal. Acepta los términos solo si estás de acuerdo.

Iniciar el emulador en Android Studio

1. Abre Android Studio. No necesitas abrir QueVer ahí.
2. Desde la bienvenida: **More Actions → Virtual Device Manager** (en algunas versiones aparece como Device Manager).
3. Si tienes un proyecto abierto: **Tools → Device Manager**.
4. Busca el dispositivo que ya usan en clase y pulsa ▶. Si ya está encendido, no lo inicies otra vez.

Resultado esperado: se abre una ventana de teléfono. Espera a que aparezca la pantalla de Android y puedas desbloquearla.

Si falla: si no hay dispositivos en la lista, consulta al expositor antes de descargar una imagen. Si sigue arrancando, espera; no pulses ▶ de nuevo.

VS Code · terminal integrada · comprobar el emulador

```
adb devices
```

Resultado esperado: una salida similar a esta. El número del emulador puede variar.

```
List of devices attached
emulator-5554    device
```

Si falla: si la lista está vacía, inicia el emulador desde Android Studio. Si dice `offline`, espera a que termine de arrancar y repite el comando.

VS Code · terminal integrada · ejecutar desde quever/

```
cordova run android --emulator
```

Resultado esperado: la compilación termina con `BUILD SUCCESSFUL` y Cordova instala y abre su app inicial en el emulador. La primera vez puede tardar por las descargas.

Si falla: si la compilación termina pero no abre la app, revisa `adb devices`. Si aparece `BUILD FAILED`, revisa el primer error y la sección Resolver errores.

Editar en VS Code

1. En el Explorador, expande `quever`. Mantén abierta la carpeta `taller-cordova`; no necesitas cambiar de ventana.
2. Edita `quever/www/` y `quever/config.xml`. No edites `platforms/android/`.
3. Para crear un archivo, pulsa con el botón derecho en su carpeta → New File. Escribe el nombre indicado en la fase.
4. Para reemplazarlo: ⌘A, pega el archivo completo con ⌘V y guarda con ⌘S.

Resultado esperado: el archivo aparece en la ruta indicada. Al guardar, desaparece el punto de cambios pendientes de su pestaña. La app aún no cambia: debes volver a ejecutarla.

Si falla: revisa la carpeta y el nombre exactos; evita duplicados como `index.js.txt`. Si abriste otra terminal, vuelve a configurar sus variables y entra a `quever/`.

Después de cada fase, guarda los archivos y ejecuta `cordova run android --emulator` desde `quever/`. No uses Live Server: abrir el HTML en un navegador no ejecuta los plugins Android.

Crear la pantalla y la base de datos

Esta fase muestra la pantalla inicial y prepara SQLite. Aún no incluye el buscador.

Cómo funciona

La WebView carga el HTML. `cordova.js` conecta JavaScript con Android. El evento `deviceready` indica cuándo podemos usar los plugins. Después, `initDB()` crea la tabla `movies` si no existe. Si ya existe, conserva sus datos.

Archivos

Las rutas parten de tu carpeta `quever/`. En `www/js/config.local.js`, cambia `TU_API_KEY` por tu clave OMDb entre comillas. Si aún no la tienes, deja el marcador.

Prueba

Guarda y ejecuta `cordova run android --emulator`.

1. Debe aparecer “QueVer” y “La base de tu catálogo está lista”.
2. Espera el mensaje “Tu catálogo está listo para recibir películas”. La tabla ya se creó.
3. Cierra y abre la app. El mensaje debe reaparecer.

La tabla incluye los campos de las siguientes fases. Las funciones de guardado se añaden después.

Consultar OMDb

Añade la búsqueda y el detalle de películas. Necesitas internet y tu clave OMDb activada.

Cómo funciona

El formulario dispara `submit`. `searchMovies()` lee el título y `requestOmdb()` consulta OMDb con `fetch`. La respuesta llega en JSON; `renderSearchResults()` la muestra. “Ver detalle” hace otra consulta con el identificador IMDb. Todavía no guardamos nada en SQLite.

> Conserva `www/js/config.local.js`. Si aún tiene `TU_API_KEY`, escribe tu clave antes de probar.

Prueba

Guarda y ejecuta `cordova run android --emulator`.

1. Busca **Inception**. Deben aparecer resultados.
2. Pulsa “Ver detalle”. Comprueba título, año y sinopsis.
3. Busca sin escribir. Debe pedirte un título.
4. Si aparece “Invalid API key”, revisa la clave y el correo de activación.

Guardar una película

Añade las opciones “La quiero ver” y “Ya la vi”. Para las vistas, guarda también calificación y opinión.

Cómo funciona

`saveMovie()` reúne los datos de OMDb, el estado y tu opinión. `insertMovie()` los envía al plugin SQLite. Al terminar, consultamos la base y actualizamos el listado. Los `?` de SQL representan valores que enviamos por separado. `imdb_id` es único para evitar duplicados.

Prueba

Guarda y ejecuta `cordova run android --emulator`.

1. Busca una película, abre su detalle y pulsa “La quiero ver”. Debe aparecer en el listado.
2. Guarda otra con “Ya la vi”, calificación y opinión.
3. Intenta guardar la primera otra vez. Debe avisar que ya existe y conservar sus datos.
4. Cierra y abre QueVer. Las películas deben seguir guardadas.

No desinstales la app: perderías su base local.

Separar vistas y pendientes

Muestra las películas en tarjetas, separadas por estado, y permite abrir su detalle.

Cómo funciona

`getMoviesByStatus()` consulta SQLite por estado. `loadCatalog()` muestra las dos secciones, con hasta cinco películas en cada una. “Ver más” abre el resto de la lista local; no consulta OMDb.

Prueba

Guarda y ejecuta `cordova run android --emulator`.

1. Las películas anteriores deben aparecer en su sección.
2. Abre una tarjeta y comprueba sus datos y tu opinión.
3. Con más de cinco películas en un estado, “Ver más” debe mostrar el resto. Con cinco o menos, no aparece.

Actualizar los datos guardados

Permite editar la opinión, marcar una pendiente como vista y eliminar una película.

Cómo funciona

Completamos el CRUD: `INSERT` crea, `SELECT` consulta, `UPDATE` modifica y `DELETE` elimina. Usamos el identificador IMDb para elegir el registro. La pantalla se actualiza después de que SQLite confirma la operación.

Prueba

Guarda y ejecuta `cordova run android --emulator`.

1. Edita la opinión o calificación de una película vista. Abre su detalle otra vez y comprueba el cambio.
2. Marca una pendiente como vista. Debe cambiar de sección.
3. Pulsa “Eliminar” y cancela. La película debe permanecer.
4. Repite y confirma. Debe desaparecer, incluso después de reiniciar la app.

Añadir una recomendación aleatoria

Añade el botón “Recomiéndame una” y permite guardar su resultado.

Cómo funciona

Elegimos una palabra al azar, consultamos OMDb y seleccionamos uno de sus resultados. Después cargamos su detalle. Para guardarlo, `saveRandomMovie()` llama a `saveMovie()`, la misma función de las fases anteriores. La selección es aleatoria, no personalizada.

Prueba

Guarda y ejecuta `cordova run android --emulator`.

1. Pulsa “Recomiéndame una”. Debe mostrar una película y opciones de guardado.
2. Guárdala como pendiente o vista y comprueba el catálogo.
3. Cierra otra recomendación sin guardar. El catálogo no debe cambiar.
4. Sin internet, debe avisar del error sin abrir un cuadro vacío.

Prueba final

1. Busca una película y guárdala.
2. Edita su opinión y cierra la app desde las aplicaciones recientes de Android.
3. Ábrela desde su icono. Comprueba que conserva la película y tu cambio.
4. Desactiva internet. El catálogo guardado debe funcionar; la búsqueda y las recomendaciones, no.
5. Explica qué ejecuta la WebView, qué hace el plugin y dónde se guardan los datos.

▶ Opcional: instalarla en tu teléfono Android

Resolver errores

▶ **¿En qué carpeta ejecuto Cordova?**

▶ **No reconoce node, npm o cordova / error EACCES**

▶ **Java incorrecto o ANDROID_HOME no encontrado**

▶ **Falta target Android, Build-Tools, sdkmanager o licencias**

▶ **No aparece el emulador o hay varios destinos**

▶ **Gradle descarga mucho, tarda o falla**

▶ **SQLite no disponible o pantalla “Preparando…”**

▶ **La búsqueda falla o dice “Invalid API key”**

▶ **Guardé archivos pero la pantalla no cambia / me atrasé**

▶ **Ver el error en la consola**
